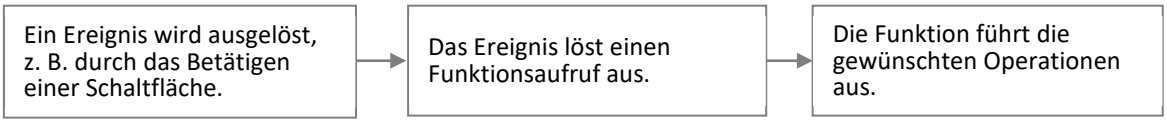
9

Ereignisse

9.1 Grundlagen zu Ereignissen

Es gibt die Möglichkeit, dass JavaScript-Anweisungen sofort nach dem Laden einer Webseite ausgeführt werden. Der Aufruf dieser Anweisungen wird direkt in den Skript-Bereich geschrieben. In der Praxis wird diese Möglichkeit nur für Initialisierungen und Deklarationen genutzt. Der Regelfall ist, dass Anweisungen erst nach Eintritt eines sogenannten **Ereignisses** abgearbeitet werden. Ein Ereignis erfolgt beispielsweise bei einem gezielten Aufruf einer Funktion durch einen Anwender (z. B. Anwender klickt eine Schaltfläche) oder einer maschinellen Aktion (z. B. Server schickt Daten an Browser). In JavaScript können Sie gezielt auf zahlreiche Ereignisse reagieren.

Tritt ein bestimmtes Ereignis ein, wird die angegebene Funktion aufgerufen, und die enthaltenen Anweisungen werden abgearbeitet.



Eine Webseite interaktiv zu gestalten, bedeutet also, auf die Aktivitäten des Benutzers und gewisser maschineller Ereignisse zu reagieren. Ein typisches Beispiel ist das Auswerten eines HTML-Formulars. JavaScript ermöglicht dabei das Abfangen von fehlerhaften Eingaben, bevor der Formularinhalt an den Webserver geschickt wird. Dies hilft, unnötigen Datentransfer im Internet zu vermeiden, die Server nicht durch Fehleingaben zu belasten und dem Benutzer Wartezeiten zu ersparen. Andere Aktivitäten sind z. B. vom Benutzer ausgeführte Mausbewegungen und -klicks.

Im **D**ocument **O**bject **M**odell (DOM) besitzen alle sichtbaren HTML-Objekte Ereignisse (engl.: events) als Eigenschaften, auf die der Browser mit der Ausführung von JavaScript-Anweisungen reagieren kann.

Schulversion
© HERDT-Verlag

9.2 Ereignisbehandlung

Zu einer Reaktion auf die verschiedenen Ereignisse, die in einem Browser verwertet werden können, verwenden Sie **Eventhandler**. Das Verfahren heißt **Ereignisbehandlung**.

Eventhandler

Ein Eventhandler-Ereignis wird in JavaScript als Eigenschaft von allen Objekten zur Verfügung gestellt, für die ein Ereignis überwacht werden kann. Das sind einmal die DOM-Objekte, aber es können auch unsichtbare Objekte sein, die etwa den Datenaustausch mit einem Server im Hintergrund verwalten (was man unter Ajax und manchen neuen HTML5-Techniken macht).

Die Eventhandler, die Sie für Knoten einer Webseite unter JavaScript verwenden, gehören direkt zum DOM-Konzept und nicht zu JavaScript selbst. Sie werden oft auch JavaScript-Eventhandler genannt und im Buch unter diesem Begriff verwendet. Für Objekte, die **nicht** zur DOM-Schnittstelle zählen, gehören die Eventhandler zu JavaScript. Als Programmierer brauchen Sie diese Details nicht zu berücksichtigen, denn der Zugriff auf einen Eventhandler erfolgt in jedem Fall wie üblich über die Punktnotation.

```
document.onclick=meinefunktion;
```

Im Beispiel sehen Sie den Zugriff auf eine Eigenschaft `onclick` eines Objekts mit Namen `document`. Die Eigenschaft ist ein Eventhandler – eine Zuweisung einer Funktionsreferenz zu einer passenden Eigenschaft des Objekts, bei dem das Auslösen von dem Ereignis überwacht werden soll. Alternativ können Sie auch eine anonyme Funktion verwenden, zum Beispiel:

```
document.onclick=function() { … };
```

In der Praxis werden in der Regel anonyme Funktionen verwendet.

Über die verschiedenen Versionen des DOM-Konzepts wurden immer wieder neue Eventhandler hinzugefügt. Die Eventhandler der neueren DOM-Versionen können in der Praxis nur mit Vorsicht und genauer Überprüfung des Browsers eines Besuchers verwendet werden. Nachfolgend werden nur die Eventhandler angegeben, die in der Praxis sinnvoll sind.

HTML-Eventhandler

Früher hat man oft in Webseiten Eventhandler als Attribute in einen HTML-Tag notiert. Beispiel:

```
<h3 onclick='MeineFunktion()'>Überschrift</h3>
```

Diese Möglichkeit besteht weiterhin, sollte aber nicht verwendet werden. Nur mit JavaScript-Eventhandlern können Sie eine **saubere Trennung von Struktur und Funktionalität** gewährleisten. Die Vermischung von JavaScript-Aufrufen direkt in HTML-Tags verhindert diese Trennung. JavaScript-Eventhandler können sowohl in einen internen Skriptcontainer als auch in externe JavaScript-Dateien ausgelagert werden. Dazu kommt, dass es für jeden HTML-Eventhandler ein JavaScript-Gegenstück gibt. Es gibt jedoch Ereignisse, für die kein HTML-Eventhandler zur Verfügung steht.

© HERDT-Verlag Schulversion

9.3 Auf Ereignisse reagieren

Die verfügbaren Ereignisse, auf die Sie unter JavaScript reagieren können, kann man nach Kategorien unterteilen.

Fensterereignisse und Dokumentereignisse

Ein aktives Browserfenster stellt Ereignisse bereit, die zum `window`-Objekt gehören. Hier finden Sie eine Auflistung der wichtigsten Ereignisse:

Ereignis	Wird ausgelöst …
`onbeforeload`	vor dem Laden der Webseite
`onblur`	wenn das Fenster den Fokus verliert
`onfocus`	wenn das Fenster den Fokus erhält
`onload`	beim Laden einer Webseite
`onunload`	beim Verlassen einer Webseite

Die wichtigen Ereignisse sind die Reaktion auf das Laden und Verlassen der Webseite. Sie werden mit `onload` und `onunload` behandelt. Die Eventhandler werden direkt beim `window`-Objekt notiert.

Beispiel

```
window.onload=Funktionsname;
```



Die Eventhandler könnten theoretisch auch bei anderen Elementen wie Grafiken verwendet werden. Diese Möglichkeit wird in der Praxis nicht genutzt. Es gibt im aktuellen DOM-Konzept ebenso einige weitere Dokumentereignisse (`onabort`, `onerror`, `onresize` oder `onscroll`), die bei vielen noch in der Praxis eingesetzten Browsern nicht funktionieren und deren Verwendung nicht zu empfehlen ist.

Fehler beim Zugriff auf den DOM

Der Eventhandler `onload` wird zum Schutz des DOM-Baums genutzt. Dieser Schutz ist besonders wichtig. Es ist in JavaScript ein großes Problem, wenn Sie auf Bestandteile des DOM-Baums zugreifen, bevor der DOM vollständig aufgebaut ist.

© HERDT-Verlag

Beispiel: *kap9/falscher_dom_zugriff.html*

Angenommen, Sie wollen eine Grafik manipulieren, bevor der Browser deren DOM-Objekt erstellt hat. Der folgende Code macht den Fehler deutlich.

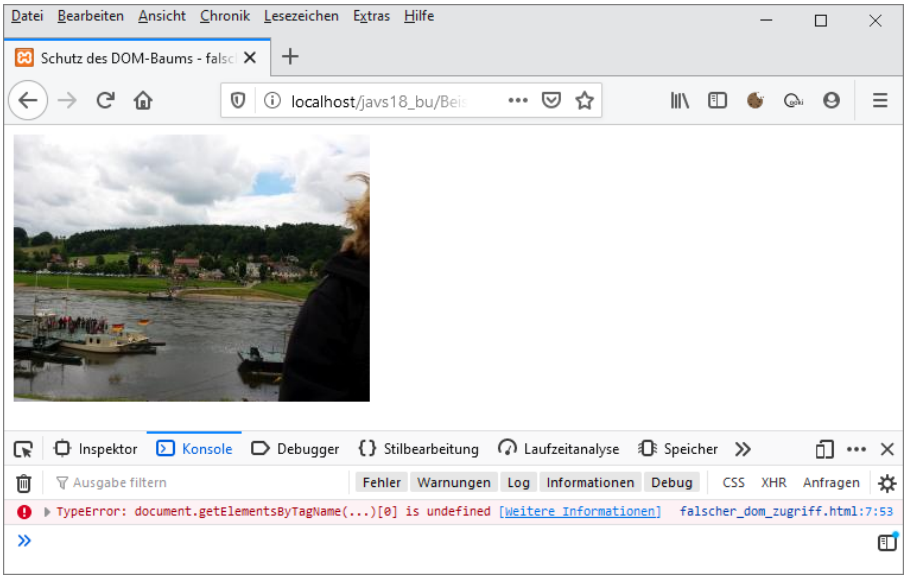
```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Schutz des DOM-Baums - falscher Zugriff</title>
  <script type="text/javascript">
①   document.getElementById("info").innerHTML =
②     document.getElementsByTagName("img")[0].alt;
  </script>
</head>
<body>
③ <img src="images/b1.jpg" alt="Am Fluss"/>
④ <div id="info"></div>
</body>
</html>
```

① Mit `document.getElementById("info")` wird in dem Skript beim Laden der Webseite auf das DOM-Objekt zugegriffen, das den ›div‹-Container repräsentiert, der erst später in der Webseite definiert wird. Das kann zu dem Zeitpunkt nicht funktionieren, und das Skript bricht mit einem Fehler ab.

② Mit `document.getElementsByTagName("img")[0]` wird auf das erste Bild in der Webseite zugegriffen. Zu dem Zeitpunkt gibt es im DOM noch kein Objekt, das dieses Bild repräsentiert. Auch das kann nicht funktionieren! Es gibt somit zwei fehlerhafte DOM-Zugriffe in dem Beispiel.

③ Hier wird erst das Bild per HTML definiert, und der Browser kann das DOM-Objekt aufbauen.

④ Nun wird erst das DOM-Objekt für den ›div‹-Container erzeugt. Die JavaScript-Aufrufe, die auf diese DOM-Objekte zugreifen, könnten erst jetzt durchgeführt werden.



Fehlermeldung in der Web-Konsole bei falschen DOM-Zugriffen

Eine Initialisierungsfunktion mit `onload`

DOM-Zugriffe dürfen nicht zu früh verwendet werden. Früher wurden deshalb der Skriptbereich bzw. die Skriptreferenz mit den Zugriffen auf DOM-Objekte erst **am Ende der Webseite eingebunden**, zum Beispiel:

```
<!DOCTYPE html>
<html>
<head>
 …
</head>
<body>
  // normale Webseite
① <script type="text/javascript" src="…"></script>
</body>
</html>
```

① Die Referenz auf die externe Skript-Datei oder den Skript-Container mit dem DOM-Zugriff ist am Ende des Body-Bereichs notiert.

Diese Möglichkeit besteht heute immer noch. Eleganter können Sie das Problem mit der Initialisierungsfunktion des Eventhandlers `onload` lösen. Der Eventhandler `onload` erlaubt eine Initialisierungsfunktion zu erstellen, wobei der Eventhandler in einigen alten Browsern (insbesondere in älteren Versionen des Internet Explorers) fehlerhaft implementiert ist. Die Probleme sind in neueren Browsern nicht mehr vorhanden. Die Notation von Skripten am Ende der Webseite findet man derzeit dennoch wieder häufiger – wenngleich aus anderen Gründen. Man kann damit unter gewissen Konstellationen das Laden relevanter Bestandteile einer Webseite vorziehen gegenüber später benötigten Bestandteilen.

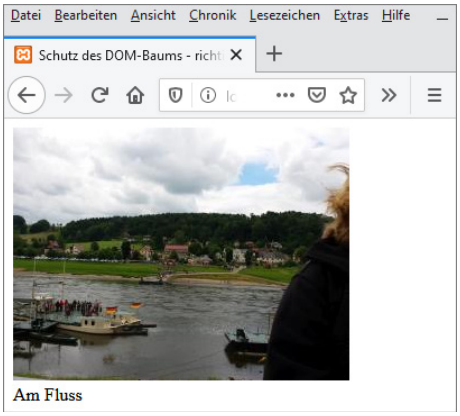
Die folgende Abwandlung des letzten Beispiels zeigt den Weg, wie Sie heutzutage beim Laden einer Webseite auf den DOM zugreifen.

```
  <script type="text/javascript">
…
①   window.onload=function() {
②     document.getElementById("info").innerHTML =
        document.getElementsByTagName("img")[0].alt;
    }
  </script>
</head>
…
```

① **Alle** DOM-Zugriffe, die beim Laden der Webseite ausgeführt werden, werden in einer Funktion durchgeführt, die mit `onload` erst nach dem Fertigstellen des DOM-Baums aufgerufen wird. Das gewährleistet den Schutz des DOM-Baums.

② Im Inneren der Funktion kann man nun gefahrlos mit `document.getElementById("info")` auf den >div<-Container und mit `document.getElementsByTagName("img")[0]` auf das erste Bild in der Webseite zugreifen.

© HERDT-Verlag



Nun funktioniert der Zugriff

Mausereignisse

Klicks auf Referenzen mit der Maus sowie das Überstreichen eines Elements einer Webseite mit dem Mauszeiger sind sehr oft genutzte Ereignisse, um ein JavaScript aufzurufen. In der nachfolgenden Tabelle sehen Sie Ereignisse, auf die Sie reagieren können und die in der Praxis breite Unterstützung in den aktuellen Browsern haben:

Event	Wird ausgelöst, wenn …
`onclick`	ein Klick in den Bereich eines Elements erfolgt
`ondblclick`	ein Doppelklick in den Bereich eines Elements erfolgt
`onmousedown`	eine Maustaste in dem Bereich eines Elements gedrückt wird
`onmouseenter`	die Maus in den Bereich eines Elements eintritt
`onmouseleave`	die Maus den Bereich eines Elements verlässt
`onmousemove`	der Mauszeiger bewegt wird, solange er sich im Bereich eines Elements befindet
`onmouseout`	die Maus den Bereich eines Elements oder eines enthaltenen Kindelements verlässt
`onmouseover`	der Mauszeiger bewegt wird, solange er sich im Bereich eines Elements oder eines enthaltenen Kindelements befindet
`onmouseup`	eine Maustaste in dem Bereich eines Elements wieder losgelassen wird

Beispiel

```
document.getElementsByTagName("h1")[0].onclick=Funktionsname;
```

Es gibt eine Reihe von Eventhandlern, die auf Drag & Drop-Aktionen (Verschieben und Loslassen von Elementen) reagieren (`ondragstart`, `ondrag`, `ondragenter`, `ondragleave`, `ondragover`, `ondrop`, `ondragend`). Obwohl diese Eventhandler erst mit DOM 2 eingeführt wurden, werden sie sogar umfangreich in Browsern unterstützt. Allerdings ist die eigentliche Programmierung von Drag & Drop-Aktionen nicht trivial, und deshalb werden Sie vermutlich selten mit diesen Eventhandlern in Kontakt kommen.

Beispiel für einen Rollover-Effekt: *kap9/bildwechsel.html*

Ein Einsatzgebiet der beiden Ereignisse `onmouseout` und `onmouseover` ist die gemeinsame Verwendung bei einem Rollover-Effekt. Rollover bedeutet, dass eine Grafik sich in dem Moment ändert, in dem die Maus über sie bewegt wird. Verlässt die Maus die Grafik, kehrt die Grafik zur ursprünglichen Anzeige zurück. Diesen Effekt finden Sie in den Navigationsbereichen von Webseiten.

Das Beispiel zeigt, wie Sie einen einfachen Rollover-Effekt erzeugen. Dabei werden zwei gleich große Grafiken gegeneinander ausgetauscht.

```
  <script type="text/javascript"><…
①   window.onload=function() {
②     document.images[0].src="images/b3.jpg";
③       document.getElementById("b1").onmouseover=function() {
④         document.images[0].src="images/b1.jpg";
        }
⑤       document.getElementById("b1").onmouseout=function() {
          document.images[0].src="images/b2.jpg";
      }
    }
  </script>
</head>
<body>
  <h3>onmouseover und onmouseout</h3>
  <img alt="Grafikwechsel" title="Grafikwechsel" id="b1" />
</body>
</html>
```

① Da auf den DOM zugegriffen werden soll, wird mit `window.onload` eine anonyme Initialisierungsfunktion erstellt. Das Bild selbst ist mit HTML in der Webseite vorhanden, die `src`-Eigenschaft wurde explizit **nicht** mit HTML gesetzt.

② Das Bild wird per JavaScript gefüllt, indem die Bildeigenschaft `src` mit dem Wert `"images/b1.jpg"` gefüllt wird.



Die Webseite nach dem Laden

© HERDT-Verlag

③ Das Bildobjekt, das eine ID `b1` hat, wird mit `document.getElementById()` angesprochen, und beim Eventhandler `onmouseover` wird eine anonyme Funktion registriert.

④ Das Bild auf der Webseite wird durch das neue Bild ersetzt.

⑤ Analog wird für den Eventhandler `onmouseout` vorgegangen. Die für den Rollover-Effekt verantwortlichen `onmouseover`- und `onmouseout`-Handler werden gemeinsam beim Bildobjekt registriert. Beim Überfahren mit der Maus (`onmouseover`) wird die eine Grafik angezeigt, beim Verlassen mit der Maus (`onmouseout`) die andere Grafik.



Die Grafik bei `onmouseout`



Die Grafik bei `onmouseover`

Tastaturereignisse

Die Reaktion auf das Betätigen einer (beliebigen) Taste auf der Tastatur ist eine weitere Situation, auf die Sie reagieren müssen. Man unterscheidet drei verschiedene Ereignisse.

Event	Wird ausgelöst, wenn …
`onkeydown`	eine Taste gedrückt wird.
`onkeyup`	eine Taste losgelassen wird.
`onkeypress`	eine Taste gedrückt gehalten wird. Nach dem Drücken einer Taste steht der Tastaturcode in einem Tastaturpuffer zur Verfügung.

Formularereignisse

Es gibt eine Reihe von Ereignissen, die im Zusammenhang mit Webformularen sinnvoll sind. Das sind zum einen die Änderung, die Aktivierung und das Verlassen eines Elements in einer Webseite. Diese Aktionen sind bei Elementen zur Entgegennahme von Benutzereingaben sinnvoll. Zum anderen geht es um das Verschicken von Formulareingaben bzw. das Zurücksetzen des Formulars.

Event	Wird ausgelöst, wenn …
`onblur`	ein Element den Fokus verliert
`onchange`	ein Element geändert wurde
`onfocus`	ein Element den Fokus bekommt

Event	Wird ausgelöst, wenn …
`onreset`	ein Formular zurückgesetzt wird
`onselect`	ein Element selektiert wird
`onsubmit`	ein Formular abgeschickt wurde

9.4 Das Ereignisobjekt `event`

Eine Eigenschaft des `window`-Objekts ist das `event`-Objekt (event = Ereignis). Mit diesem Objekt können Sie auf Ereignisse wie Mausklicks oder Tastatureingaben reagieren und JavaScript-Code ausführen, der bestimmte Informationen in dem Ereignisobjekt verwendet. Beispielsweise können Sie bei einem Mausklick ermitteln, ob die linke oder rechte Maustaste gedrückt wurde, oder bei einem Tastendruck die betätigte Taste ermitteln.

Das Ereignisobjekt bzw. `event`-Objekt, das hinter der Ereignisbehandlung in JavaScript liegt, ist ein Mitteilungsobjekt mit Informationen über die Art des ausgelösten Ereignisses. Solche Ereignisobjekte werden vom Browser unablässig erzeugt, und ein solches Objekt kann bei der Reaktion auf Ereignisse gezielt verwertet werden. Dabei dürfen Sie nicht nur an die wenigen offensichtlichen Ereignisse wie den Klick des Besuchers mit der Maus oder eine Tastatureingabe denken. Das einfache Verschieben des Mauszeigers über den Bereich des Browsers findet quasi permanent statt.

Dabei können in kürzester Zeit Tausende von Ereignisobjekten entstehen. Jedes Verschieben einer Mauszeigerposition um wenige Pixel bewirkt in der Regel bereits das Erzeugen von einem eigenständigen Ereignisobjekt. So kann beispielsweise das Scrollen eines Textes um wenige Millimeter bereits mehrere Ereignisobjekte generieren.

Das Ereignisobjekt stellt zur gezielten Reaktion eine Reihe interessanter Eigenschaften mit spezifischen Informationen wie auch diverse Methoden bereit. Betrachten Sie als Beispiel die Situation, wenn ein Anwender mit der Maus in irgendeinen Bereich der Webseite klickt. Bei einem Mausklick erzeugt der Browser ein Ereignisobjekt, das u. a. folgende Informationen enthält:

- die verwendete Maustaste
- eventuell gedrückte Zusatztasten
- die Koordinaten des Klicks

Andere Ereignisobjekte, die bei weiteren Ereignissen erzeugt werden, beinhalten natürlich andere Informationen, die dem Ereignis angepasst sind. So steht beispielsweise bei einem Tastendruck die gedrückte Taste als abzufragende Information bereit. Allgemein beinhaltet ein Ereignisobjekt zahlreiche sinnvolle Informationen, die Sie zur Erstellung gut angepasster Applikationen nutzen können.

© HERDT-Verlag

Event-Bubbling

Es gibt im Zusammenhang mit der Behandlung des Ereignisobjekts den Begriff der **Bubble-Events** bzw. der Bubble-Phase. Wenn ein Ereignis bei einem Knoten in einer Baumstruktur wie dem DOM-Baum auftritt, stellt sich die Frage, welches der Objekte im Baum nun für ein aufgetretenes Ereignis zuständig ist. Knoten sind im Baum sehr tief ineinander verschachtelt. Wenn ein Anwender auf ein Bild in einer Webseite klickt, hat er ja auch gleichzeitig auf die Webseite selbst geklickt. Ist die Webseite oder das Bild für die Behandlung zuständig? Und wie erfährt der Knoten der Webseite ggf. davon, dass auf den untergeordneten Knoten des Bildes ein Klick erfolgt ist, der dort nicht behandelt wurde? Oder was soll passieren, wenn mehrere ineinander geschachtelte Objekte den gleichen Eventhandler besitzen und deshalb auf das Ereignis reagieren könnten?

Die Probleme werden über das sogenannte **Event-Bubbling** gelöst. Ein Ereignis wird bei diesem Konzept immer zuerst im **innersten** Element behandelt, bei dem es aufgetreten ist – sofern es dort einen geeigneten Eventhandler gibt! Ein behandeltes Ereignisobjekt wird nach der Handhabung in einem Eventhandler sofort vernichtet.

Das innerste Element ist in der Hierarchie des DOM-Baums am weitesten von der Wurzel entfernt. Hat dieses Element keinen geeigneten Eventhandler, wird das Ereignisobjekt an das nächsthöhere Objekt im DOM-Baum (dem direkten Vorfahren) weitergeleitet usw. Es steigt wie eine Blase (Bubble) über alle Vorfahren hinauf bis zur Wurzel. Es „blubbert“ durch den Baum, bis es behandelt wird.

Wenn es bis zur Wurzel keine Behandlung von dem Ereignisobjekt gab, wird das Ereignisobjekt bei einer Variante des Event-Bubblings dort vernichtet. Allerdings gibt es für das Event-Bubbling auch eine zweite Variante, bei der ein unbehandeltes Ereignisobjekt nach Erreichen der Wurzel wieder bis zum auslösenden Element zurückblubbert und auf dem Weg zurück behandelt werden kann. Dazu werden Sie eine geeignete Methode kennenlernen.

Dieses Bubbling der Ereignisse können Sie in alten Internet Explorern mit der Eigenschaft `cancelBubble` unterbinden, indem Sie den Wert auf `true` setzen. In allen anderen Browsern wird zum Unterbinden von Bubbling die Methode `stopPropagation()` aufgerufen.

9.5 Das Ereignisobjekt verwenden

Um das Ereignisobjekt zu verwenden, mussten Sie bis vor wenigen Jahren zwei inkompatible Modelle zur Ereignisbehandlung unterscheiden:

- die alte Ereignisbehandlung von Microsoft
- die vom W3C standardisierte Ereignisbehandlung

Das standardisierte Konzept für die Reaktion per JavaScript auf das Auftreten von einem Mitteilungsobjekt innerhalb des Browsers geht in vielen Aspekten auf die Firma Netscape zurück. Das Modell wird mittlerweile von allen modernen Browsern unterstützt (auch vom Internet Explorer ab der Version 9) und das proprietäre Microsoft-Modell spielt in der Praxis keine Rolle mehr.

© HERDT-Verlag
Schulversion

Der entscheidende Unterschied beider Modelle ist, dass im Standardmodell der **erste Parameter** einer Ereignisfunktion immer das `event`-Objekt repräsentiert.

Beispiel für die anonyme Behandlung:

```
document.getElementsByTagName("h1")[0].onclick = function(evt) {
  // evt ist das event-Objekt in der Funktion –
     eine lokale Variable
};
```

Beispiel für die benannte Behandlung mit einer Funktionsreferenz:

```
document.getElementsByTagName("h1")[0].onclick =
         benannteEreiginsfunktion;
…
function benannteEreiginsfunktion (evt) {
  // evt ist das event-Objekt in der Funktion
};
```

Im alten Microsoft-Modell hingegen wird kein Parameter verwendet, sondern es gibt in jeder Ereignisfunktion standardmäßig eine lokale Variable `event`, die das `event`-Objekt repräsentiert. Aber darauf soll hier nicht mehr eingegangen werden.



Beide Ereignismodelle beinhalten einige veraltete Eigenschaften und Methoden, die in modernen Browsern entweder nicht mehr unterstützt werden oder nicht eingesetzt werden sollten, z. B. die Methode `captureEvents()`. Die Verwendung dieser alten Techniken ist unnötig, denn die hier vorgestellten Techniken beinhalten alles, was man braucht.

Das standardisierte Ereignismodell

Im standardisierten Ereignismodell gibt es folgende Eigenschaften:

Eigenschaft	Bedeutung
`offsetX`, `offsetY`	Koordinaten des Ereignisses bezogen auf das auslösende Objekt oder Element
`pageX`, `pageY`	Koordinaten des Ereignisses bezogen auf den Dokumentbereich des Browsers
`screenX`, `screenY`	Koordinaten des Ereignisses bezogen auf den gesamten Bildschirm
`target`	Referenz auf das eigentliche Zielobjekt des Ereignisses
`type`	Zeichenkette mit dem Namen des Ereignisses. Der Name des Ereignisses entspricht dem Namen des auslösenden Eventhandlers, wenn Sie das "on" weglassen, z. B. "`click`" bei einem `onclick`-Eventhandler.
`which`	Ganzzahl, die bei Maus-Ereignissen den Button, bei Tasten den ASCII-Code der Taste repräsentiert. Der Wert 1 steht bei einem Mausereignis für die linke Maustaste, 2 für die mittlere und 3 für die rechte Maustaste.

© HERDT-Verlag

Beispiel: *kap9/tastaturauswertung_standard.html*

Hier folgt ein Beispiel zur Auswertung des Tastaturcodes nach dem standardisierten Ereignismodell. In diesem Beispiel soll die folgende einfache Situation realisiert werden: Der Besucher drückt eine beliebige Taste, und der Tastaturcode der gedrückten Taste wird mittels `getElementById().innerHTML` in der Webseite angezeigt.

```
<script type="text/javascript">
① window.onkeypress = function(ev) {
②   document.getElementById("antwort").innerHTML =
      "Gedrückter Tastencode: " + ev.which;
  }
</script>
</head>
<body>
    <h1>Drücken Sie eine beliebige Taste!
    </h1><div id="antwort"></div>
</body>
```

① Im Skript wird das Ereignis `onkeypress` beim `window`-Objekt überwacht (Drücken irgendeiner Taste). Wenn der Anwender eine Taste drückt, wird die anonyme Callback-Funktion aufgerufen. Nach dem Niederdrücken einer Taste wird der Tastaturcode in dem Tastaturpuffer zur Verfügung gestellt.

② Der erste Parameter, der an die Ereignisfunktion übergeben wird, ist in dem Standard-ereignismodell per Vorstellung **immer** eine Referenz auf das Ereignisobjekt. Die Eigenschaft `which` des Ereignis-Objekts enthält den Tastaturcode. In der Funktion wird der aktuelle Wert bei jedem Tastendruck in dem `<div>`-Element ausgegeben (der numerische Code, der über die Eigenschaft `which` verfügbar ist).



Eine Taste wurde gedrückt

Die Methode `addEventListener()` als Alternative zu Eventhandlern

Im standardisierten Ereigniskonzept gibt es eine zentrale Funktion, mit der man auf Ereignisobjekte reagieren kann – ein sogenannter **Event Listener**, wie er in Sprachen wie Java oder C# üblich ist. Die Methode heißt `addEventListener()`. Die Methode erwartet drei Parameter:

- Der erste Parameter gibt an, welcher Ereignistyp überwacht werden soll. Diese werden im Wesentlichen wie die Eventhandler benannt, nur entfällt das vorangestellte **on**, z. B. `click`, `mouseover`, `mousedown`, `mouseup`, `mousemove` oder `keyup`. Die Ereignisse werden als Strings notiert.
- Der zweite Parameter ist die Funktion, die beim Eintreten des Events aufgerufen werden soll.
- Der dritte Parameter gibt an, ob das Ereignis beim Aufsteigen in der Bubble-Phase aufgefangen werden soll (`true`) oder erst beim Weg zurück.

Beispiel: *kap9/tastaturauswertung_addeventlistener.html*

Hier ist ein Beispiel mit der Methode `addEventListener()`, in der der Tastaturcode ausgewertet und der Anzeigebereich direkt nach dem Loslassen der Taste wieder gelöscht wird. Sie müssen somit auf zwei Ereignisse reagieren.

```
<script type="text/javascript"><…
① window.addEventListener("keypress",function(ev) {
    document.getElementById("antwort").innerHTML =
      "Gedrückter Tastencode: " + ev.which;
  },true);
② window.addEventListener("keyup",function(ev) {
    document.getElementById("antwort").innerHTML = "";
  },true);
</script>
</head>
<body>
    <h1>Drücken Sie eine beliebige Taste!
    </h1><div id="antwort"></div>
</body>
```

① Die Ereignismethode wird beim `keypress`-Event für das gesamte Browserfenster registriert. Nach dem Drücken einer Taste wird der Tastaturcode im Tastaturpuffer zur Verfügung gestellt und in der Ereignisfunktion verwertet, die als zweiter Parameter angegeben wird.

② Eine weitere Ereignismethode wird registriert, allerdings beim `keyup`-Event, das beim Loslassen einer Taste ausgelöst wird. Wenn das Ereignis ausgelöst wird, wird in der Ereignisfunktion der Anzeigebereich im Browser wieder geleert.

Schulversion
© HERDT-Verlag

9.6 Übungen

Übung A: Theoriefrage zu Events

Übungsdatei: -- **Ergebnisdatei:** *kap09/uebung1.pdf*

1. Erklären Sie, welche Benutzeraktionen mit den folgenden Eventhandlern verbunden sind:
   - `onabort`
   - `onchange`
   - `onclick`
   - `onmouseout`
   - `onfocus`
   - `onload`
   - `onunload`

Übung B: Reagieren auf Benutzeraktionen

Übungsdatei: -- **Ergebnisdatei:** *kap09/uebung2.html*

1. In dieser Übung sollen Sie Informationen zu einem Mausklick nach dem Standard-ereignismodell auswerten. Die Ereignisbehandlung wird per Funktionsreferenz an das `document`-Objekt gebunden. Auf das Loslassen der Maustaste soll reagiert werden (`document.onmouseup`). Zu dem Zeitpunkt stehen spezifische Informationen über den Mausklick zur Verfügung.
2. Bei dem überwachten Eventhandler `onmouseup` soll eine Funktionsreferenz die Funktion `pos()` referenzieren. In der Funktion setzen Sie einen Antworttext mit diversen Werten aus dem Ereignisobjekt nach dem Standardereignismodell zusammen. Diese sollen in einem `<div>`-Container in der Webseite mit `innerHTML` ausgegeben werden.

© HERDT-Verlag Schulversion